

From AI-Generated Programs to AI-Assisted Software Architects

Ted Lehr

Associate Professor of Practice, Computer Science
Texas State University

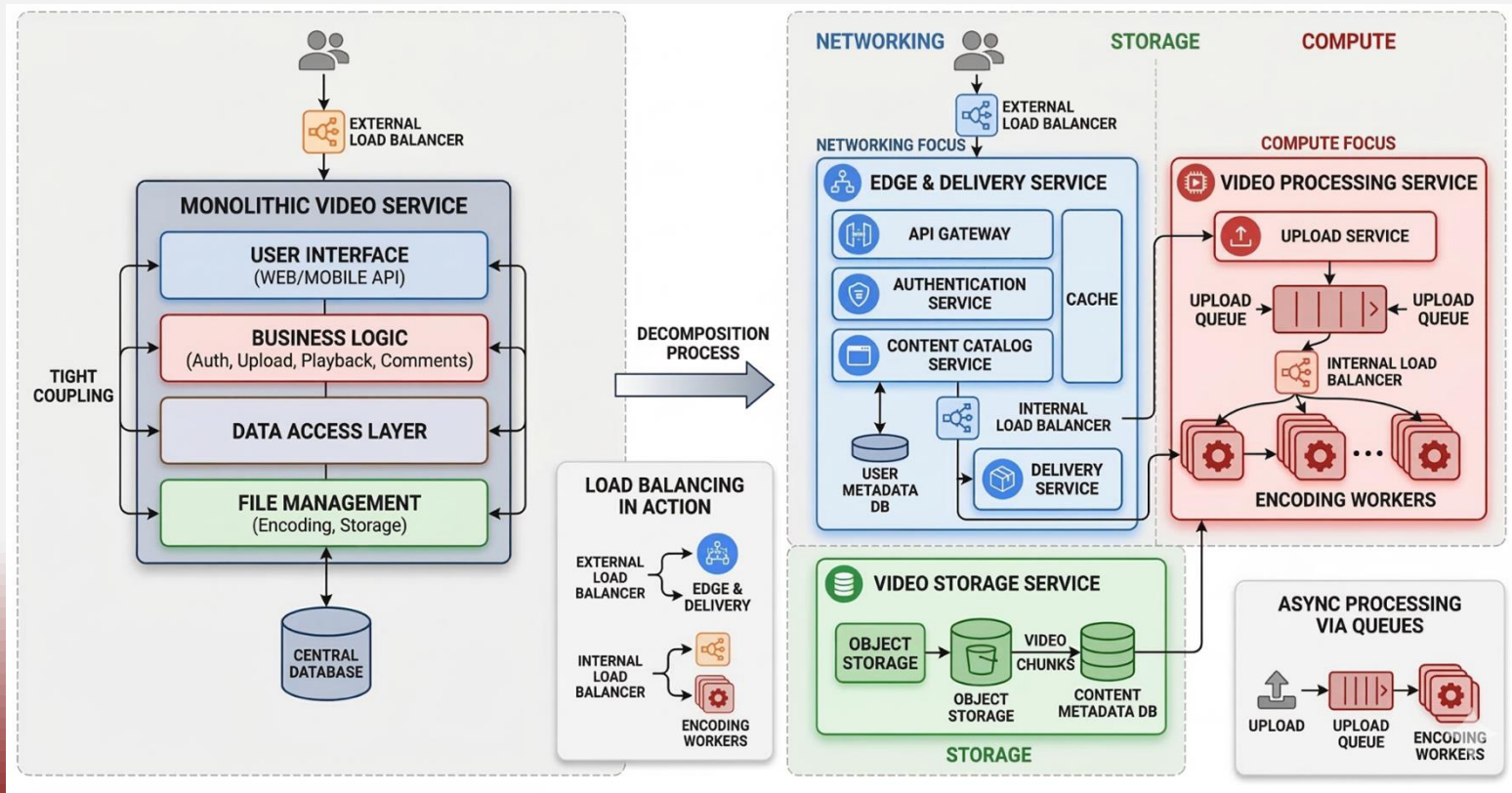
How AI Changed My
Java and Software
Engineering Courses

An Observation

- AI has largely solved code generation.
 - Pause while the laughter subsides.
- It has not solved architecture.

Typical Architecture generated by
Student and AI

An architecture preferred by
Industry



I encourage students to use AI

- Students now create or extend 10–100 file desktop applications, cloud apps with AI assistance.
- The applications use APIs, databases, threading and micro-services.

Project Structure

```
stockrush/  
├── client/  
│   └── src/  
│       ├── pages/  
│       ├── components/  
│       ├── services/  
│       ├── types/  
│       └── utils/  
├── server/  
│   └── src/  
│       ├── controllers/  
│       ├── middleware/  
│       ├── routes/  
│       └── services/  
├── database/  
│   ├── migrations/  
│   └── seeds/  
├── docs/  
└── research/
```

← Frontend (React + TypeScript + Tailwind)
← Page-level views (Dashboard, Markets, etc.)
← Reusable UI components (Navbar, PriceCard, etc.)
← API call functions (authService, marketService)
← TypeScript interfaces and type definitions
← Validators and helper functions
← Backend (Node.js + Express)
← Route handler logic
← Auth middleware (JWT verification)
← Express route definitions
← External API adapters + Redis cache
← SQL migration scripts
← Test data seed scripts
← Architecture docs and API contracts
← Sprint research documents

Tech Stack

Frontend

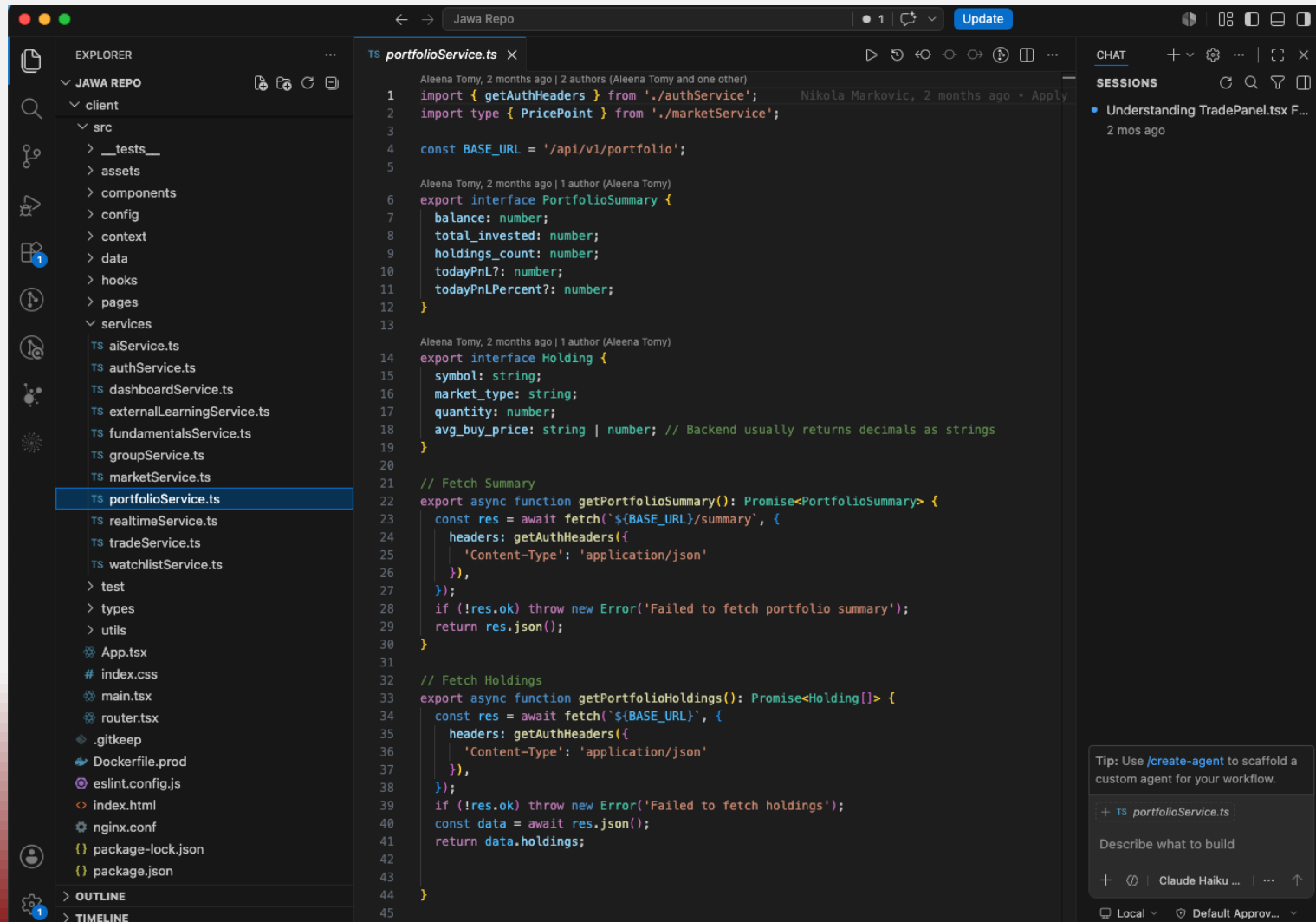
Technology	Version	Purpose
React	19.2	UI framework
TypeScript	~5.9	Type safety
Vite	7.3	Build tool & dev server
React Router	v7	Client-side routing
Recharts / Chart.js	3.8 / latest	Data visualization
dnd-kit	latest	Drag-and-drop scheduling
vite-plugin-pwa	0.19	Progressive Web App / offline caching

Backend

Technology	Version	Purpose
FastAPI	latest	Async HTTP framework
SQLModel	latest	ORM (SQLAlchemy + Pydantic)
Alembic	latest	Database migrations
PostgreSQL	16	Primary database
Uvicorn	latest	ASGI server
python-jose + bcrypt	latest	JWT auth + password hashing
scikit-learn	≥1.4	ML demand forecasting
joblib	≥1.3	ML model persistence

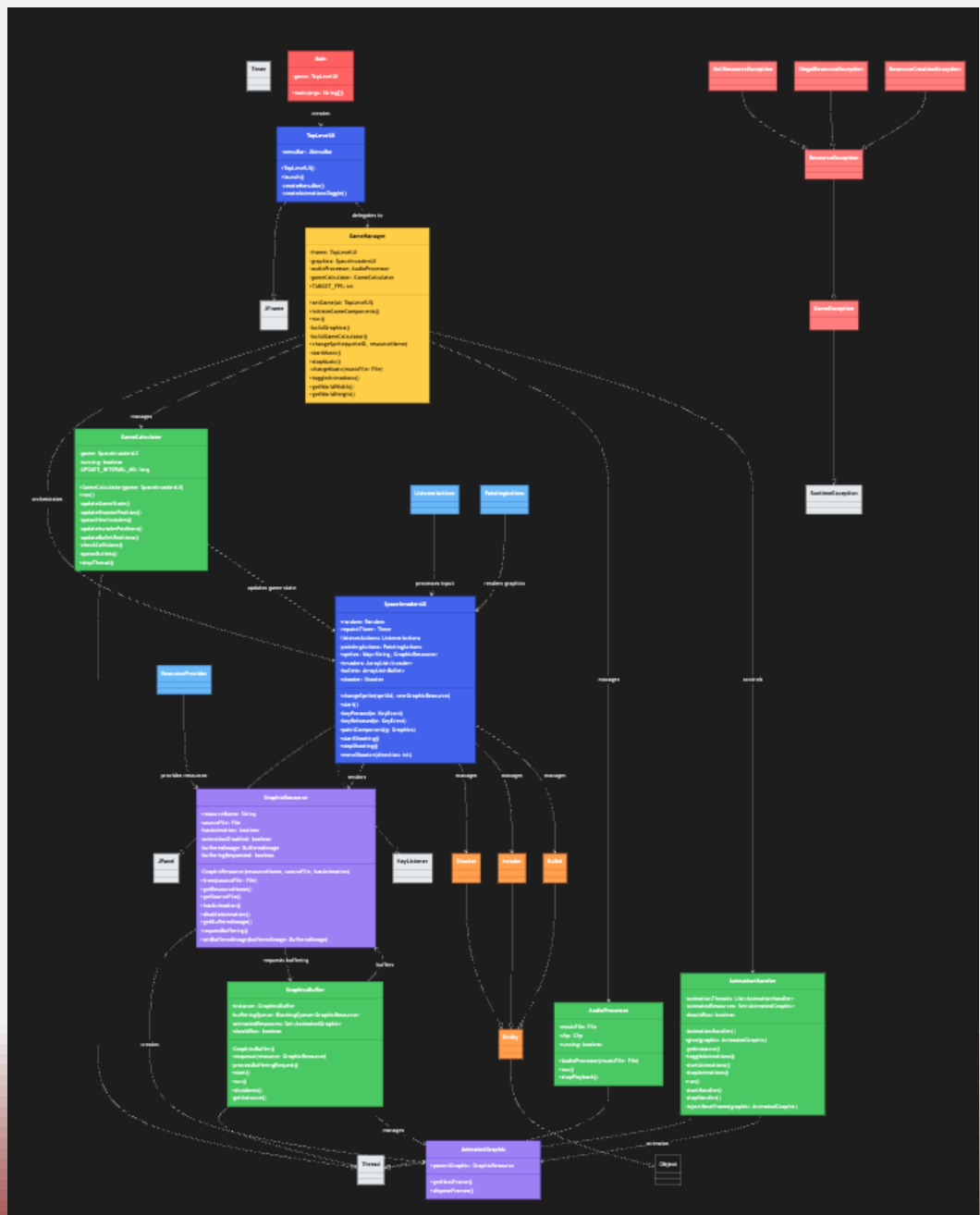
What AI Does Well

- Code generation, CRUD operations, unit tests, API integration, documentation, and boilerplate.



What AI Does Poorly

- architecture,
- simplicity,
- concurrency,
- race conditions,
- scalability,
- performance bottlenecks,
- maintainability.



Static architectures don't 'fit' dynamic loads

What I Changed in My Courses

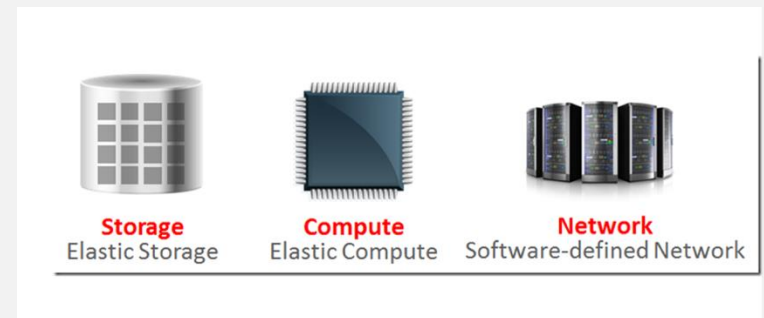
- I “met the students where they are.” I encourage them to use AI.
 - I learn from them on how they use it. They learn from me about it limits vis-à-vis production quality code.
- Assignments used to assume or accept a single user of a application. Now I require support for multi-user, multi-threaded, and in the case of cloud applications, micro-services.
 - I’m adding ‘just-in-time’ scalability requirements.
 - I’m considering asking other students to take-over assignments from the original students.

Responsibility-Oriented Concurrency

(Single Responsibility Principle, Separation of Concerns)

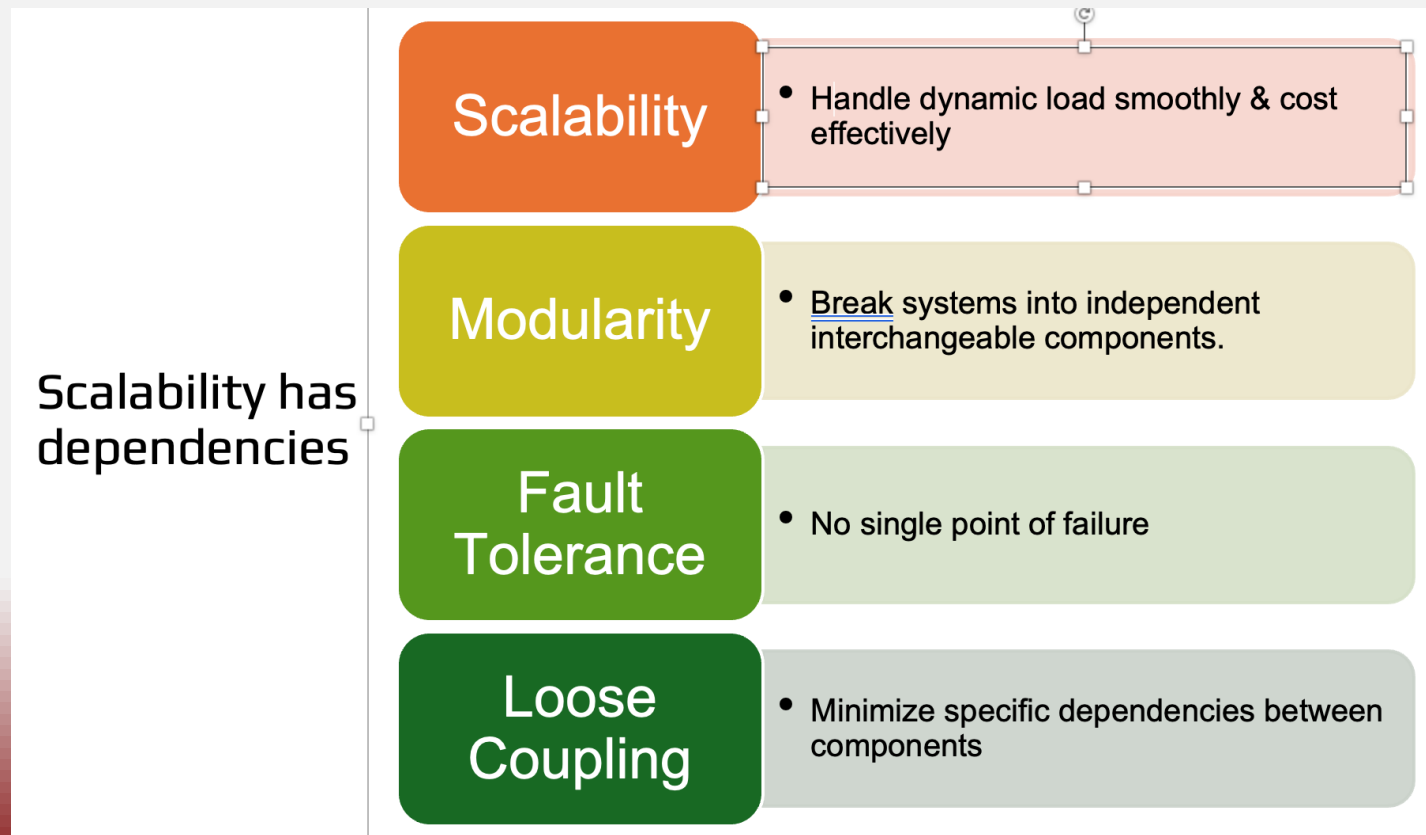
- Separate concerns into network, application and storage.
 - queues for communication
 - worker threads for
 - persistence (storage),
 - computation logic,
 - logging,
 - analytics,
 - notifications,
 - request handling.

Single Responsibility: The Big Three



Understanding how well the Student and the AI have done

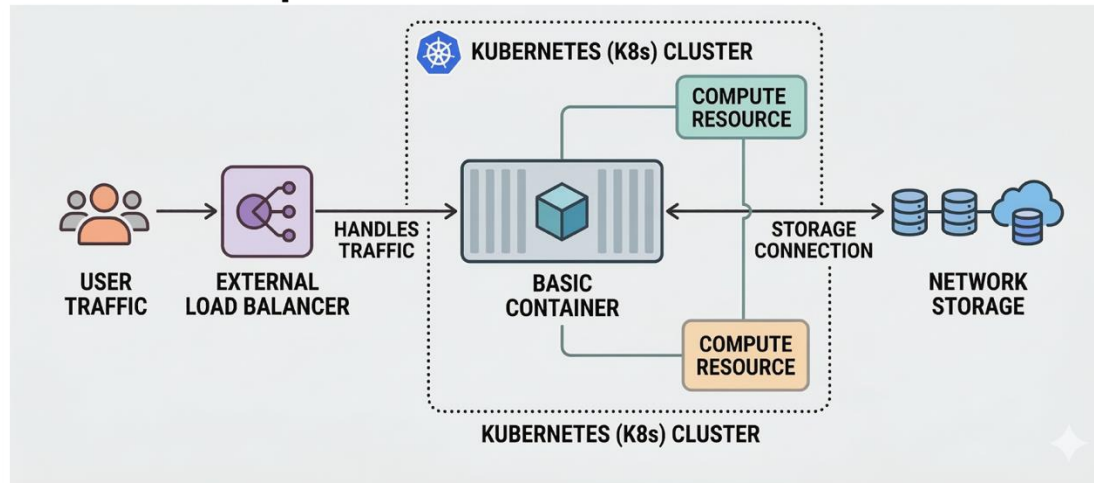
- Students will use load generators, measure performance, queue lengths, and bottlenecks.



Automatic Scaling Experiments

- Architectures detect either:
 - Actually detect overloaded components and spawn additional workers for the affected responsibility.
 - Set load (user, service request, etc) thresholds that spawn additional workers for the affected responsibility whether or not a performance degradation is detected

Start Simple



What Students Learn

- Architecture, systems thinking, performance engineering, observability, and AI supervision and evaluation.

Conclusion and Discussion

- The question is no longer: Can students write code? The question is: Can they design the system?
- How should computer science education evolve when students can generate large codebases instantly?